

Plano de Testes — ServeRest Front (Login)

---	---
Projeto	serverest-front-playwright
Módulo/Feature sob teste	Tela de Login — https://front.serverest.dev/login
Responsável	Rodrigo Sousa — QA Automation Engineer / SDET
Ferramentas	Playwright, Node.js, GitHub Actions, Playwright MCP + Claude Code
Versão do documento	1.0
Data	Setembro/2026

1. Introdução

Este documento descreve o plano de testes para a funcionalidade de ****autenticação (login)**** do front-end do ServeRest, uma aplicação React disponibilizada publicamente para fins de estudo e prática de automação de testes. O plano cobre o escopo, a estratégia, os critérios de entrada/saída, os cenários de teste, os riscos e os artefatos de automação associados.

2. Objetivo

Garantir que o fluxo de login do ServeRest funcione corretamente para os principais caminhos felizes, caminhos de exceção e regras de validação de campo, com cobertura automatizada de regressão executada a cada alteração no código (via CI).

3. Escopo

3.1 Dentro do escopo

- ? Autenticação com credenciais válidas
- ? Autenticação com credenciais inválidas (email não cadastrado, senha incorreta)
- ? Validações de campo obrigatório (email, senha, ambos)
- ? Validação de formato de email
- ? Navegação a partir da tela de login (link "Cadastre-se")

3.2 Fora do escopo (não coberto por este plano)

- ? Fluxo de cadastro de usuário (`/cadastrarusuarios`)
- ? Recuperação/redefinição de senha (funcionalidade não exposta na tela de login do ServeRest)

- ? Testes de API (`POST /login` da API REST do ServeRest) — **planejado como próxima fase**, reaproveitando a suíte Postman já existente no portfólio
- ? Testes de performance/carga
- ? Testes de acessibilidade (a11y) formais
- ? Testes visuais (visual regression)
- ? Testes em dispositivos móveis reais (apenas emulação, se aplicável)

4. Itens de teste (test items)

Item	Descrição
---	---
Tela de Login	`https://front.serverest.dev/login`
Campo Email	Input de texto, obrigatório, validação de formato
Campo Senha	Input de senha, obrigatório
Botão "Entrar"	Submete o formulário
Link "Cadastre-se"	Navega para `/cadastrarusuarios`
Mensagens de validação	Exibidas em tempo real / após submissão

5. Estratégia de testes

5.1 Tipos de teste

- ? **Funcional (E2E)**: fluxo completo de login via interface, ponta a ponta, automatizado com Playwright.
- ? **Validação de UI**: mensagens de erro e de campo obrigatório.
- ? **Regressão**: toda a suíte roda automaticamente em cada push/PR via CI.

5.2 Técnicas de design de casos de teste

- ? **Particionamento de equivalência**: classes de email/senha válidos vs. inválidos.
- ? **Análise de valor limite**: campos vazios vs. preenchidos.
- ? **Tabela de decisão**: combinações de email/senha (ambos vazios, só um vazio, ambos preenchidos e inválidos, ambos válidos).

5.3 Abordagem de automação

- ? **Framework**: Playwright (JavaScript), com padrão **Page Object Model** (`tests/pages/LoginPage.js`) para isolar seletores da lógica de teste.

? **Massa de dados**: usuário público de exemplo do ServeRest para o caminho feliz; `@faker-js/faker` para dados aleatórios em cenários negativos (evita colisão de massa entre execuções).

? **Execução cross-browser**: Chromium, Firefox e WebKit.

? **CI/CD**: GitHub Actions executa a suíte a cada push/PR na branch `main` e publica o relatório HTML como artefato.

? **Manutenção assistida por IA**: uso do Playwright MCP conectado ao Claude Code para validar seletores contra o DOM real e gerar novos cenários (documentado em `MCP-CLAUDE-WORKFLOW.md`).

6. Ambiente de teste

Item	Detalhe
---	---
Ambiente sob teste	Produção pública (<code>front.serverest.dev</code>)
Navegadores	Chromium, Firefox, WebKit (via <code>playwright.config.js</code>)
Execução local	Node.js 18+, <code>npm test</code> / <code>npm run test:ui</code>
Execução em CI	Ubuntu (GitHub Actions runner), Node.js 20
Dados de teste	Usuário público <code>fulano@qa.com</code> (caminho feliz) + dados gerados via <code>faker</code> (caminhos negativos)

7. Critérios de entrada

? Ambiente `front.serverest.dev` acessível e estável.

? Estrutura do projeto (`package.json`, `playwright.config.js`) configurada.

? Seletores da `LoginPage` validados contra o DOM real (ver seção 11 — riscos).

8. Critérios de saída

? 100% dos casos de teste do escopo (seção 9) executados.

? Nenhuma falha aberta classificada como bloqueante (login válido não funcionar, ou suíte não rodar em CI).

? Pipeline de CI verde na branch `main`.

? Relatório HTML gerado e revisado.

9. Cenários e casos de teste

Rastreabilidade entre caso de teste, arquivo de automação e prioridade.

ID	Cenário	Prioridade	Automatizado em
----	---------	------------	-----------------

---	---	---	---
CT-01	Login com credenciais válidas redireciona para `/home`	Alta	`login.spec.js`
CT-02	Login com email não cadastrado exibe "Email e/ou senha inválidos"	Alta	`login.spec.js`
CT-03	Login com senha incorreta para email válido exibe a mesma mensagem de erro	Alta	`login.spec.js`
CT-04	Email e senha vazios exibem as duas validações obrigatórias	Média	`login.spec.js`
CT-05	Apenas email vazio exibe validação do campo email	Média	`login.spec.js`
CT-06	Apenas senha vazia exibe validação do campo senha	Média	`login.spec.js`
CT-07	Email com formato inválido não autentica	Média	`login.spec.js`
CT-08	Clique em "Cadastre-se" navega para `/cadastrarusuarios`	Baixa	`login.spec.js`

10. O que NÃO será testado (e por quê)

Item excluído	Justificativa
---	---
Testes de API do endpoint `/login`	Escopo deste plano é a camada de UI; API será tratada em plano/suíte à parte, reaproveitando o projeto Postman já existente
Recuperação de senha	Funcionalidade não presente na tela de login atual do ServeRest
Testes de carga/performance	Fora do objetivo de regressão funcional deste projeto
Visual regression	Não há baseline de design definida; poderia gerar falsos positivos

11. Riscos e mitigação

	Impacto	Mitigação
	---	---
o na criação do projeto)	Alto — testes podem falhar por seletor errado, não por bug real	Validar com `playwright codegen` ou Playwright MCP
	Médio — massa de dados pode colidir ou o ambiente pode estar instável/fora do ar	Uso de `faker` para dados únicos nos cenários negativos
	Médio — quebra de seletores sem relação com o objetivo do teste	Rodar a suíte periodicamente via CI; usar o fluxo de M
	Baixo	Revisar falhas específicas de browser antes de report

12. Entregáveis

- ? Código-fonte da suíte de automação (`tests/`)
- ? Este documento de Plano de Testes (`docs/TEST\_PLAN.md`)
- ? Pipeline de CI (`.github/workflows/playwright.yml`)
- ? Relatório de execução HTML (gerado a cada run, publicado como artefato do CI)
- ? Guia de manutenção assistida por IA (`MCP-CLAUDE-WORKFLOW.md`)

13. Papéis e responsabilidades

Papel	Responsável
---	---

Elaboração do plano e automação	Rodrigo Sousa
Execução em CI	GitHub Actions (automático)
Manutenção de seletores/novos cenários	Rodrigo Sousa, com apoio do Playwright MCP + Claude Code

14. Cronograma (referencial)

Fase	Descrição	Status
---	---	---
1	Levantamento do escopo e desenho dos cenários	Concluído
2	Implementação da automação (POM + specs)	Concluído
3	Configuração de CI	Concluído
4	Validação dos seletores contra o DOM real via MCP	Pendente
5	Extensão para testes de API do login	Planejado

15. Aprovação

Este é um projeto de portfólio pessoal; não há fluxo formal de aprovação por stakeholders externos. A suíte é considerada "aprovada" quando o critério de saída (seção 8) é atendido e o pipeline de CI está verde.